

# Chromatin Loop Calling and Differential Analysis

## Overview

This notebook reproduces the **chromatin loop calling and enhancer–promoter contact analysis** from Wang, R., Lee, J.-H., Kim, J. *et al.* “SARS-CoV-2 restructures host chromatin architecture.” *Nature Microbiology* **8**, 679–694 (2023). <https://doi.org/10.1038/s41564-023-01344-8>

The paper shows that SARS-CoV-2 infection preferentially depletes cohesin from intra-TAD regions, weakening chromatin loops and reducing enhancer–promoter contacts at interferon response gene loci. We quantify and test these changes using a re-implementation of the HIC-CUPS loop-calling algorithm available in **cooltools**, followed by DESeq2 differential testing and Aggregate Peak Analysis (APA).

## Deviations from the paper

The analysis shown in the paper looked at genome wide chromatin loops at 5kb and 10kb resolution. We focused our analysis on chromosome 14 and were limited to analysing loops at 10kb resolution.

---

## 0. Data Preparation

The same files and preprocessing steps as in the `tad analysis` were used here.

```

mkdir -p ./out

cooler merge ./out/merged_mock_HiC.mcool \
  ./GSE179184_RAW/mock_HiC_rep1.mcool::resolutions/10000 \
  ./GSE179184_RAW/mock_HiC_rep2.mcool::resolutions/10000

cooler merge ./out/merged_cov_HiC.mcool \
  ./GSE179184_RAW/cov_HiC_rep1.mcool::resolutions/10000 \
  ./GSE179184_RAW/cov_HiC_rep2.mcool::resolutions/10000

cooler balance ./out/merged_mock_HiC.mcool
cooler balance ./out/merged_cov_HiC.mcool

cooler zoomify -r 10000 \
  ./out/merged_mock_HiC.mcool \
  -o ./out/merged_mock_HiC_multi.mcool

cooler zoomify -r 10000 \
  ./out/merged_cov_HiC.mcool \
  -o ./out/merged_cov_HiC_multi.mcool

```

---

## 1. Loop Calling with cooltools dots

We use `cooltools dots` (the open2C re-implementation of HICCUPS (Rao *et al.*, 2014)) to detect pixels in the Hi-C contact matrix that are significantly enriched relative to a donut-shaped local expected background. Loop calling is performed on the merged replicate coolers to maximise read depth and sensitivity.

### 1.1 Compute Expected Values

`cooltools expected-cis` builds the background model of contact frequency as a function of genomic distance, required as input to `cooltools dots`.

```

# chr14 size in hg19 = 107,043,718 bp
echo -e "chr14\t0\t107043718\tchr14" > ./out/chr14_view.tsv

cooltools expected-cis \
  --nproc 4 \

```

```

--view ./out/chr14_view.tsv \
./out/merged_mock_HiC_multi.mcool::/resolutions/10000 \
-o ./out/expected_mock_10kb.tsv

cooltools expected-cis \
--nproc 4 \
--view ./out/chr14_view.tsv \
./out/merged_cov_HiC_multi.mcool::/resolutions/10000 \
-o ./out/expected_cov_10kb.tsv

```

## 1.2 Call Loops at 10 kb Resolution

Dots are called using the parameters specified in **4DN benchmarking protocol** (Oksuz *et al.*, 2021, *Nat. Methods*)

```

cooltools dots \
--nproc 4 \
--view ./out/chr14_view.tsv \
--fdr 0.1 \
--clustering-radius 39000 \
--max-loci-separation 10000000 \
--tile-size 5000000 \
--max-nans-tolerated 4 \
./out/merged_mock_HiC_multi.mcool::/resolutions/10000 \
./out/expected_mock_10kb.tsv \
-o ./out/loops_mock_10kb.bedpe

cooltools dots \
--nproc 4 \
--view ./out/chr14_view.tsv \
--fdr 0.1 \
--clustering-radius 39000 \
--max-loci-separation 10000000 \
--tile-size 5000000 \
--max-nans-tolerated 4 \
./out/merged_cov_HiC_multi.mcool::/resolutions/10000 \
./out/expected_cov_10kb.tsv \
-o ./out/loops_cov_10kb.bedpe

```

## 2. Build a Consensus Loop Set

Here we look for loops that are present in both infected and mock groups.

```
read_loops <- function(path, tag) {
  raw <- read_tsv(path,
    comment = "#", col_types = cols(.default = "c"),
    show_col_types = FALSE
  )

  if (!("chrom1" %in% colnames(raw))) {
    colnames(raw)[1:6] <- c(
      "chrom1", "start1", "end1",
      "chrom2", "start2", "end2"
    )
  }

  raw %>%
    mutate(across(c(start1, end1, start2, end2), as.integer)) %>%
    filter(chrom1 == "chr14", chrom2 == "chr14") %>%
    mutate(
      mid1      = (start1 + end1) / 2L,
      mid2      = (start2 + end2) / 2L,
      loop_size = mid2 - mid1,
      source    = tag
    )
}

loops_mock <- read_loops("./out/loops_mock_10kb.bedpe", "mock")
loops_cov <- read_loops("./out/loops_cov_10kb.bedpe", "cov")

margin <- 10000

anchor_gr <- function(df, anchor) {
  if (anchor == 1) {
    GRanges(df$chrom1, IRanges(start = df$mid1, end = df$mid1))
  } else {
    GRanges(df$chrom2, IRanges(start = df$mid2, end = df$mid2))
  }
}

ov1 <- findOverlaps(anchor_gr(loops_mock, 1), anchor_gr(loops_cov, 1),
  maxgap = margin
```

```

)
ov2 <- findOverlaps(anchor_gr(loops_mock, 2), anchor_gr(loops_cov, 2),
  maxgap = margin
)

shared <- inner_join(as.data.frame(ov1), as.data.frame(ov2),
  by = c("queryHits", "subjectHits")
)

cov_unique <- loops_cov %>%
  filter(!(row_number() %in% unique(shared$subjectHits)))

loops_consensus <- bind_rows(loops_mock, cov_unique) %>%
  mutate(
    loop_id      = paste0("loop_", row_number()),
    loop_size_kb = loop_size / 1e3
  )

loops_consensus %>%
  select(chrom1, start1, end1, chrom2, start2, end2, loop_id) %>%
  write_tsv("./out/loops_consensus_chr14.bedpe", col_names = TRUE)

```

---

### 3. Differential Loop Analysis

Following the paper, we quantify raw Hi-C contact counts at 40 kb resolution of each called loop, then apply DESeq2 to test for differential analysis.

#### 3.1 Prepare 40 kb Coolers

The individual replicate files contain 10 kb bins. We coarsen by a factor of 4 to reach 40 kb using `cooler coarsen`.

```

for SAMPLE in mock_HiC_rep1 mock_HiC_rep2 cov_HiC_rep1 cov_HiC_rep2; do
  cooler coarsen \
    --nproc 4 \
    -k 4 \
    ./GSE179184_RAW/${SAMPLE}.mcool::resolutions/10000 \
    -o ./out/${SAMPLE}_40kb.cool

```

```

    echo "Coarsened: ${SAMPLE}_40kb.cool"
done

```

### 3.2 Extract Raw Contact Counts at Each Loop Locus

```

import cooler
import numpy as np
import pandas as pd

RESOLUTION = 40_000

SAMPLES = {
    "mock_rep1": "./GSE179184_RAW/mock_HiC_rep1.mcool::resolutions/10000",
    "mock_rep2": "./GSE179184_RAW/mock_HiC_rep2.mcool::resolutions/10000",
    "cov_rep1":  "./GSE179184_RAW/cov_HiC_rep1.mcool::resolutions/10000",
    "cov_rep2":  "./GSE179184_RAW/cov_HiC_rep2.mcool::resolutions/10000",
}

clrs = {name: cooler.Cooler(path) for name, path in SAMPLES.items()}

loops = pd.read_csv(
    "./out/loops_consensus_chr14.bedpe",
    sep="\t",
    header=None,
    names=["chrom1", "start1", "end1", "chrom2", "start2", "end2", "loop_id"],
)

loops = loops.drop(0)
loops[["start1", "end1", "start2", "end2"]] = loops[["start1", "end1", "start2", "end2"]].astype(int)

rows = []
for _, lp in loops.iterrows():
    m1 = int(((lp.start1 + lp.end1) // 2) // RESOLUTION * RESOLUTION)
    m2 = int(((lp.start2 + lp.end2) // 2) // RESOLUTION * RESOLUTION)

    row = {"loop_id": lp.loop_id}
    for name, clr in clrs.items():
        try:
            pixel = clr.matrix(balance=False).fetch(
                (lp.chrom1, m1, m1 + RESOLUTION),

```

```

        (lp.chrom2, m2, m2 + RESOLUTION),
    )
    row[name] = int(np.nansum(pixel))
except Exception:
    row[name] = 0
rows.append(row)

count_df = pd.DataFrame(rows)
count_df.to_csv("./out/loop_raw_counts_40kb.tsv", sep="\t", index=False)

```

### 3.3 DESeq2 Differential Loop Testing

Here we perform DESeq2 differential analysis on the raw HI-C contact counts at 40kb resolution.

```

counts_raw <- read_tsv("./out/loop_raw_counts_40kb.tsv", show_col_types = FALSE)

count_mat <- counts_raw %>%
  select(mock_rep1, mock_rep2, cov_rep1, cov_rep2) %>%
  as.matrix()
rownames(count_mat) <- counts_raw$loop_id

keep <- rowSums(count_mat) > 0
count_mat <- count_mat[keep, , drop = FALSE]

col_data <- data.frame(
  condition = factor(c("Mock", "Mock", "SARS_CoV_2", "SARS_CoV_2"),
    levels = c("Mock", "SARS_CoV_2")
  ),
  row.names = c("mock_rep1", "mock_rep2", "cov_rep1", "cov_rep2")
)

dds <- DESeqDataSetFromMatrix(
  countData = count_mat,
  colData = col_data,
  design = ~condition
)

```

converting counts to integer mode

```

dds <- DESeq(dds, quiet = TRUE)

res <- results(
  dds,
  contrast = c("condition", "SARS_CoV_2", "Mock"),
  alpha = 0.1,
  pAdjustMethod = "BH",
  independentFiltering = FALSE
)

res_df <- as.data.frame(res) %>%
  rownames_to_column("loop_id") %>%
  left_join(
    loops_consensus %>% select(loop_id, loop_size_kb),
    by = "loop_id"
  ) %>%
  mutate(
    direction = case_when(
      !is.na(padj) & padj < 0.1 & log2FoldChange > 0 ~ "Strengthened",
      !is.na(padj) & padj < 0.1 & log2FoldChange < 0 ~ "Weakened",
      TRUE ~ "NS"
    ),
    direction = factor(direction, levels = c("Weakened", "NS", "Strengthened"))
  )

n_weak <- sum(res_df$direction == "Weakened")
n_strong <- sum(res_df$direction == "Strengthened")
n_total <- nrow(res_df)

cat(sprintf(
  "Weakened loops:      %d  (%.1f%%)\n",
  n_weak, 100 * n_weak / n_total
))

```

Weakened loops: 119 (3.3%)

```

cat(sprintf(
  "Strengthened loops: %d  (%.1f%%)\n",
  n_strong, 100 * n_strong / n_total
))

```

Strengthened loops: 30 (0.8%)



```
cat(sprintf(
  "Not significant:    %d  (0.1f%%)\n",
  n_total - n_weak - n_strong,
  100 * (n_total - n_weak - n_strong) / n_total
))
```

Not significant: 3431 (95.8%)

The paper reported that genome wide 2.96% of loops were weakened and 4.7% were strengthened. We find that on chromosome 14, 3.3% are weakened and 0.8% are strengthened. This does not reflect the findings of the paper, but may be an artifact of looking solely at chromosome 14.

---

## 4. Visualisations

### 4.1 Volcano Plot

```
p_cutoff <- res_df %>%
  filter(!is.na(padj), padj < 0.1) %>%
  pull(pvalue) %>%
  max(na.rm = TRUE)

col_map <- c(
  Weakened = "#2166AC",
  NS = "grey72",
  Strengthened = "#D6604D"
)

label_map <- c(
  Weakened = sprintf("Weakened (n = %d)", n_weak),
  NS = "Not significant",
  Strengthened = sprintf("Strengthened (n = %d)", n_strong)
)

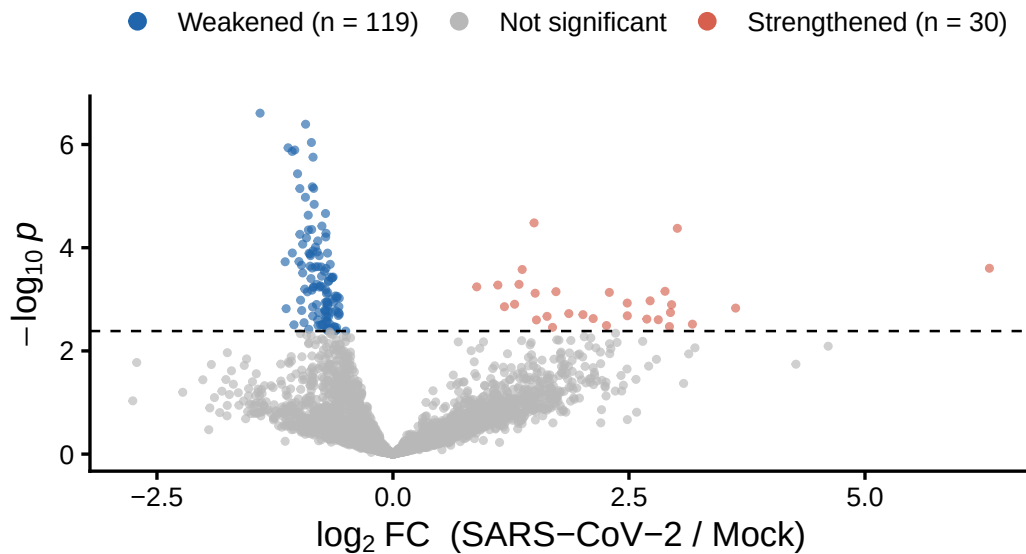
res_df %>%
  filter(!is.na(pvalue)) %>%
  arrange(direction) %>%
```

```

ggplot(aes(
  x = log2FoldChange,
  y = -log10(pvalue),
  colour = direction
)) +
geom_point(size = 0.8, alpha = 0.65) +
geom_hline(
  yintercept = -log10(p_cutoff),
  linetype = "dashed", colour = "black", linewidth = 0.45
) +
scale_colour_manual(values = col_map, labels = label_map) +
labs(
  x      = expression(log[2] ~ "FC (SARS-CoV-2 / Mock)",
  y      = expression(-log[10] ~ italic(p)),
  colour = NULL,
  title  = "Differential chromatin loops - chr14"
) +
guides(colour = guide_legend(override.aes = list(size = 2.5, alpha = 1))) +
theme_classic(base_size = 13) +
theme(legend.position = "top")

```

## Differential chromatin loops – chr14



The observed fold-changes across both weakened and strengthened loops are comparable to the changes reported in the paper (extended data figure 8). However, the ratio of strengthened

to weakened loops varies wildly between this analysis and the papers findings as previously noted.

## 4.2 Loop-Size Distribution for Differential Loops

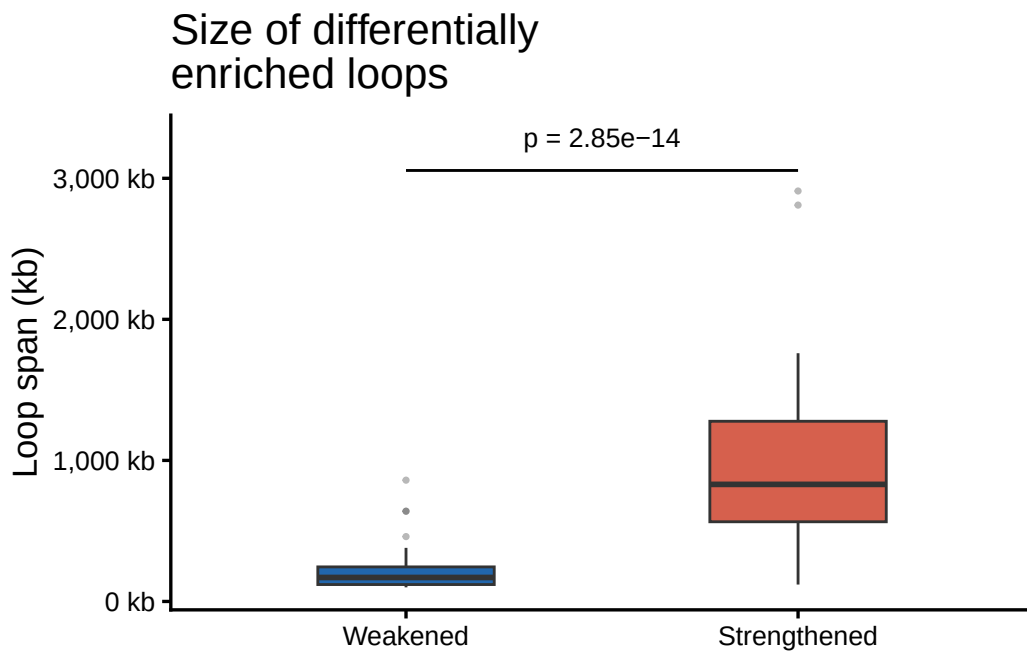
```
diff_loops <- res_df %>%
  filter(direction != "NS", !is.na(loop_size_kb)) %>%
  droplevels()

wt <- wilcox.test(
  loop_size_kb ~ direction,
  data = diff_loops,
  exact = FALSE
)

y_bar <- max(diff_loops$loop_size_kb, na.rm = TRUE)

ggplot(diff_loops, aes(x = direction, y = loop_size_kb, fill = direction)) +
  geom_boxplot(
    outlier.size = 0.6, outlier.alpha = 0.35,
    width = 0.45, linewidth = 0.5
  ) +
  annotate("segment",
    x = 1, xend = 2,
    y = y_bar * 1.05, yend = y_bar * 1.05,
    colour = "black", linewidth = 0.5
  ) +
  annotate("text",
    x = 1.5, y = y_bar * 1.13, hjust = 0.5, size = 3.5,
    label = sprintf("p = %.2e", wt$p.value)
  ) +
  scale_fill_manual(
    values = c(Weakened = "#2166AC", Strengthened = "#D6604D"),
    guide = "none"
  ) +
  scale_y_continuous(labels = comma_format(suffix = " kb")) +
  labs(
    x = NULL,
    y = "Loop span (kb)",
    title = "Size of differentially\nenriched loops"
```

```
) +  
theme_classic(base_size = 13)
```



The paper reports that virus-weakened loops are predominantly short, while strengthened loops are long and the range of different lengths is larger in strengthened loops. This is consistent with our analysis of chromosome 14.

## 5. Aggregate Peak Analysis (APA)

We perform aggregate peak analysis on all conditions and loop change types using coolpuppy.

### 5.1 Export Differential Loop Subsets

```
write_bedpe <- function(ids, file) {  
  loops_consensus %>%  
    filter(loop_id %in% ids) %>%  
    select(chrom1, start1, end1, chrom2, start2, end2, loop_id) %>%
```

```

    write_tsv(file, col_names = TRUE)
    cat(sprintf("Wrote %d loops to %s\n", length(ids), file))
}

write_bedpe(
  filter(res_df, direction == "Weakened") %>% pull(loop_id),
  "./out/loops_weakened.bedpe"
)

```

Wrote 119 loops to ./out/loops\_weakened.bedpe

```

write_bedpe(
  filter(res_df, direction == "Strengthened") %>% pull(loop_id),
  "./out/loops_strengthened.bedpe"
)

```

Wrote 30 loops to ./out/loops\_strengthened.bedpe

## 5.2 Run coolpuppy

```

# consensus loops
coolpup.py \
  --expected ./out/expected_mock_10kb.tsv \
  --view ./out/chr14_view.tsv \
  --flank 100000 \
  ./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
  ./out/loops_consensus_chr14.bedpe \
  --outname ./out/apa_mock_all.clpy

coolpup.py \
  --expected ./out/expected_cov_10kb.tsv \
  --view ./out/chr14_view.tsv \
  --flank 100000 \
  ./out/merged_cov_HiC_multi.mcool::resolutions/10000 \
  ./out/loops_consensus_chr14.bedpe \
  --outname ./out/apa_cov_all.clpy

# weakened
coolpup.py \

```

```

--expected ./out/expected_mock_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
./out/loops_weakened.bedpe \
--outname ./out/apa_mock_weakened.clpy

coolpup.py \
--expected ./out/expected_cov_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_cov_HiC_multi.mcool::resolutions/10000 \
./out/loops_weakened.bedpe \
--outname ./out/apa_cov_weakened.clpy

# strengthened
coolpup.py \
--expected ./out/expected_mock_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
./out/loops_strengthened.bedpe \
--outname ./out/apa_mock_strengthened.clpy

coolpup.py \
--expected ./out/expected_cov_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_cov_HiC_multi.mcool::resolutions/10000 \
./out/loops_strengthened.bedpe \
--outname ./out/apa_cov_strengthened.clpy

```

### 5.3 Visualise APA Heatmaps

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import re
import h5sparse

```

```
/home/louis/miniconda3/lib/R/library/reticulate/python/rpytools/loader.py:120: UserWarning: ]
11-30. Refrain from using this package or pin to Setuptools<81.
    return _find_and_load(name, import_)
```

```
import os
from matplotlib.colors import LogNorm

def load_pileup_df(filename, quaich=False, skipstripes=False):
    """
    Loads a dataframe saved using `save_pileup_df`

    Parameters
    -----
    filename : str
        File to load from.
    quaich : bool, optional
        Whether to assume standard quaich file naming to extract sample name and bedname.
        The default is False.

    Returns
    -----
    annotation : pd.DataFrame
        Pileups are in the "data" column, all metadata in other columns

    """
    with h5sparse.File(filename, "r", libver="latest") as f:
        metadata = dict(zip(f["attrs"].attrs.keys(), f["attrs"].attrs.values()))
        dstore = f["data"]
        data = []
        for chunk in dstore.iter_chunks():
            chunk = dstore[chunk]
            data.append(chunk)
        annotation = pd.read_hdf(filename, "annotation")
        annotation["data"] = data
        vertical_stripe = []
        horizontal_stripe = []
        coordinates = []
        if not skipstripes:
            try:
                for i in range(len(data)):
                    vstripe = "vertical_stripe_" + str(i)
                    hstripe = "horizontal_stripe_" + str(i)
```

```

        coords = "coordinates_" + str(i)
        vertical_stripe.append(f[vstripe][:].toarray())
        horizontal_stripe.append(f[hstripe][:].toarray())
        coordinates.append(f[coords][:].astype("U13"))
        annotation["vertical_stripe"] = vertical_stripe
        annotation["horizontal_stripe"] = horizontal_stripe
        annotation["coordinates"] = coordinates
    except KeyError:
        pass
for key, val in metadata.items():
    if key != "version":
        annotation[key] = val
if quaich:
    basename = os.path.basename(filename)
    sample, bedname = re.search(
        "^(.*)-(?:[0-9]+)_over_(.*)_(?:[0-9]+-shifts|expected).*\\.clpy", basename
    ).groups()
    annotation["sample"] = sample
    annotation["bedname"] = bedname
return annotation

def load_pileup(path):
    df = load_pileup_df(path)
    return np.nanmean(np.stack(df["data"].values), axis=0)

def plot_apa_pair(path_mock, path_cov, title_mock, title_cov):
    mat_mock = load_pileup(path_mock)
    mat_cov = load_pileup(path_cov)

    bin_size_kb = (2 * 100_000) / (mat_mock.shape[0] - 1) / 1000

    combined = np.concatenate([mat_mock.flatten(), mat_cov.flatten()])
    valid_vals = combined[(~np.isnan(combined)) & (combined > 0)]
    vmin = np.nanpercentile(valid_vals, 5)
    vmax = np.nanpercentile(valid_vals, 95)

    fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))

    n = mat_mock.shape[0]
    centre = n // 2
    flank_kb = centre * bin_size_kb
    extent = [-flank_kb, flank_kb, flank_kb, -flank_kb]

```



```

for ax, mat, title in zip(axes, [mat_mock, mat_cov], [title_mock, title_cov]):
    im = ax.matshow(mat, cmap="YlOrRd", norm=LogNorm(vmin=vmin, vmax=vmax), extent=extent)

    ax.set_title(title, fontsize=11, pad=12, fontweight="bold")
    ax.set_xlabel("Distance from center (kb)", fontsize=9)
    ax.xaxis.set_label_position('bottom')
    ax.xaxis.tick_bottom()

    ax.set_xticks([-flank_kb, 0, flank_kb])
    ax.set_yticks([-flank_kb, 0, flank_kb])

fig.subplots_adjust(left=0.08, right=0.82, bottom=0.15, top=0.85, wspace=0.35)

cbar_ax = fig.add_axes([0.86, 0.22, 0.025, 0.55])
cbar = fig.colorbar(im, cax=cbar_ax)
cbar.set_label("Enrichment (O/E)", fontsize=10, labelpad=8)

return fig

```

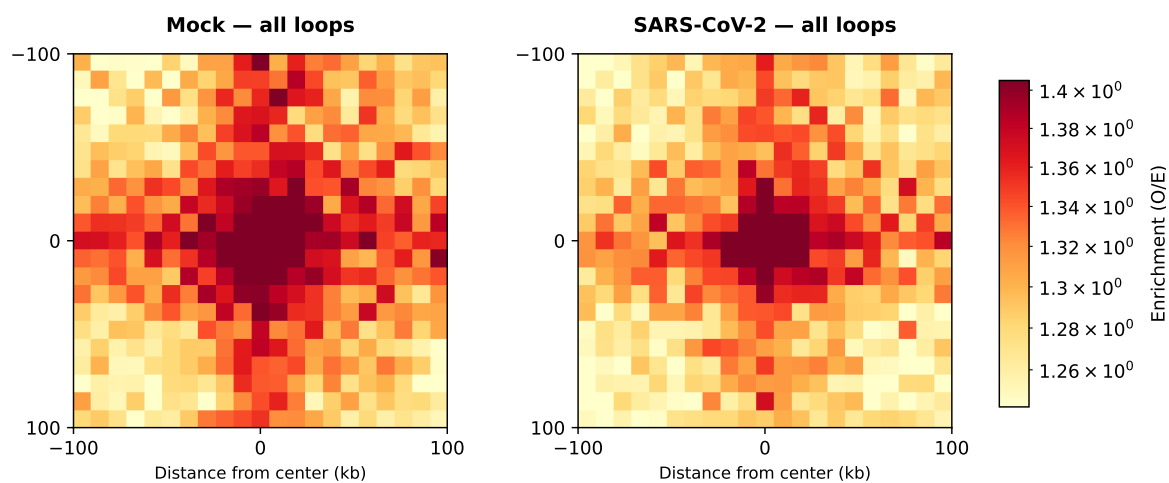
```

plot_apair(
    "./out/apa_mock_all.clpy", "./out/apa_cov_all.clpy",
    "Mock - all loops", "SARS-CoV-2 - all loops",
)

```

<Figure size 900x450 with 3 Axes>

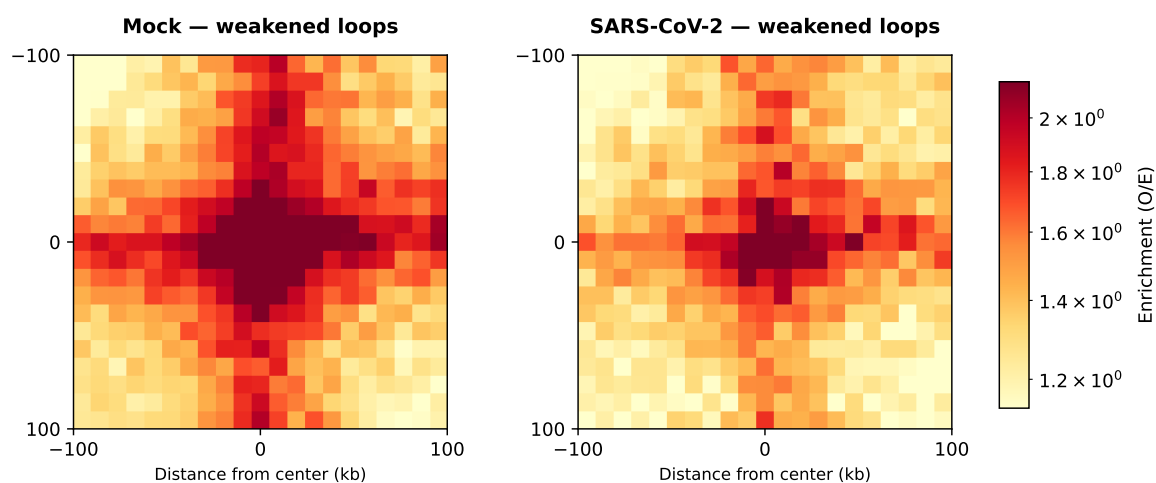
```
plt.show()
```



```
plot_apa_pair(
    "./out/apa_mock_weakened.clpy", "./out/apa_cov_weakened.clpy",
    "Mock - weakened loops", "SARS-CoV-2 - weakened loops",
)
```

<Figure size 900x450 with 3 Axes>

```
plt.show()
```



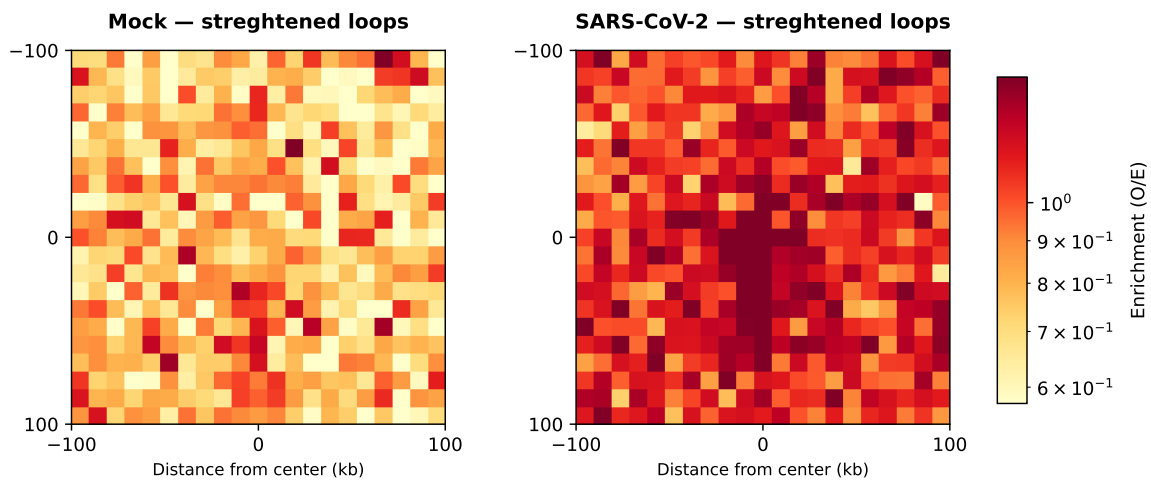
```

plot_apa_pair(
    "./out/apa_mock_strengthened.clpy", "./out/apa_cov_strengthened.clpy",
    "Mock - strenghtened loops", "SARS-CoV-2 - strenghtened loops",
)

```

<Figure size 900x450 with 3 Axes>

```
plt.show()
```



The enrichment score of all loops in mock and infected groups is comparable. Loops of the weakened category have a lower enrichment score in the infected group than in the mock group. These results match the observations in the paper.

In the APA heatmap for strengthened loops, no centralized signal is visible. While the enrichment score is higher in the infected group, which is in line with the paper's observations, the low sample size of strengthened loops makes this result unreliable.

## Discussion of our results

Since our analysis was constrained to chromosome 14 and to the analysis of loops at 10 kb resolution, the sample size of loops for our analysis was lower than in the paper. Our results do not directly replicate the results in the paper, in particular we did not find that more loops were strengthened than weakened, instead finding the reverse. However, the reported fold changes,

as well as length distributions for strengthened and weakened loops aligned with our findings. Our APA analysis also mostly supports the findings in the paper.

In conclusion we were able to show that some loops are weakened or strengthened in the infected group, but could not replicate the exact quantitative findings of the paper we tried to reproduce. This can in large parts be explained by our analysis focusing only on a subset of the experimental data.